Step 2 · 2주차 제출물 — 전처리와 특징 추출

20118701 신재윤

데이터: sEMG Hand Gesture and Elbow Angle Dataset (논문 원 데이터, 방법 A). 피험자 8명 x 손동작 4종 x 팔꿈치 각도 2종 = 8클래스, 5채널, 1,000 Hz. 1주차와 같은 방법으로 `mark=1` 지점부터 5초씩 잘라 시행 1,920개(8명 x 8클래스 x 30회)를 만든 뒤 시작했다. 코드는 `week_2.ipynb`, 실행 파일은 `scripts/preprocessing.py`, 결과는 `outputs/dataset.npz`.

① 필터 검증 — 필터 전후 스펙트럼 비교

60 Hz 노치(Q=30) + 20~450 Hz 4차 Butterworth 대역통과를 `filtfilt`로 적용했다. 아래는 피험자 a · 30_fist · 1회차 시행(5초)의 필터 전후 채널별 표준편차와 파워 스펙트럼(Welch, nperseg=1024)이다.

	bandpass 1	bandpass 2	bandpass 3	bandpass 4	bandpass 5
필터 전 표준편차	0.0208	0.0518	0.0377	0.0119	0.0049
필터 후 표준편차	0.0199	0.0501	0.0369	0.0113	0.0044

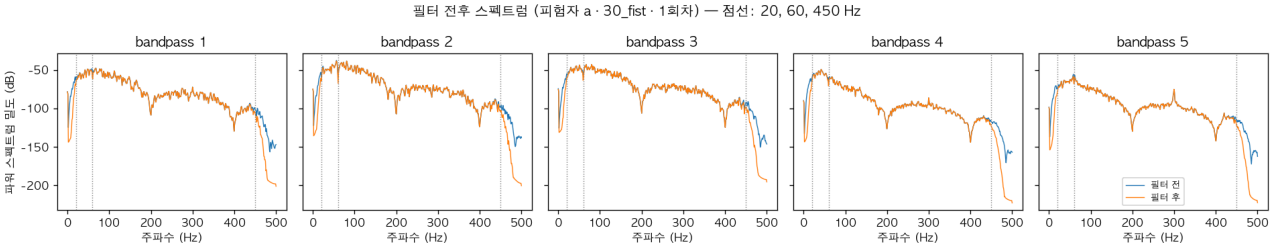


그림 1. 필터 전(파랑)과 후(주황)의 파워 스펙트럼 밀도(dB). 점선은 20, 60, 450 Hz.

20~450 Hz 안에서는 두 곡선이 완전히 겹치고, 20 Hz 아래와 450 Hz 위만 더 깎인다. 표준편차도 4% 정도만 줄었다. CSV의 열 이름이 `bandpass 1~5`이고 데이터셋 README에 같은 필터(60 Hz 노치 + 20~450 Hz 4차 Butterworth)가 이미 적용되어 있다고 적혀 있으므로, 이 결과는 데이터가 이미 필터된 상태임을 확인한 것이다. 강의 순서대로 한 번 더 적용했으며 결과에 해가 없다.

② 윈도우 확인 — 세그먼트와 윈도우 배열 모양

분석 구간은 각 시행의 1.0~3.0초(2,000샘플)로 잡았다. 강의 예시는 1.5초 시작이지만, 1주차 4-2에서 전체 시행의 평균 근활성이 1.0~2.7초에 몰려 있음을 확인했기 때문이다. 윈도우는 200 ms(200샘플), 간격 100 ms(50% 겹침)이다.

```
세그먼트 모양: (2000, 5) (1.0~3.0초)
윈도우 배열: (19, 200, 5) = (2000 − 200) // 100 + 1 = 19개 창
창 1: 1.0~1.2초 / 창 2: 1.1~1.3초 (절반이 겹침)
```

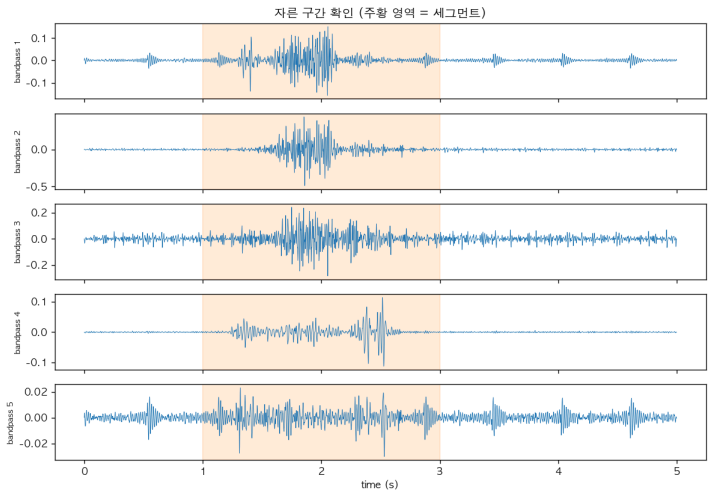


그림 2. 필터 후 시행(피험자 a · 30_fist · 1회차)에서 잘라낸 구간(주황 영역, 1.0~3.0초). 동작에 따른 근활성이 구간 안에 들어온다.

③ 특징 구현 코드 — td_features

창 하나(200, 5)에서 채널별로 시간영역 특징 5종(RMS, MAV, WL, ZC, SSC)을 구해 25차원 벡터로 만든다. 순서는 RMS 5개 → MAV 5개 → WL 5개 → ZC 5개 → SSC 5개이고, 각 5개는 bandpass 1~5 순서다.

```
FEATURE_NAMES = [f"{f}_{ch}" for f in ("RMS", "MAV", "WL", "ZC", "SSC") for ch in CH_NAMES]


def td_features(w: np.ndarray, th: float = 0.01) -> np.ndarray:
    """창 하나의 시간영역 특징. (200, 5) -> (25,)  순서는 FEATURE_NAMES와 같다"""
    rms = np.sqrt(np.mean(w ** 2, axis=0))                          # 제곱 평균의 제곱근: 신호 세기
    mav = np.mean(np.abs(w), axis=0)                                # 절댓값 평균
    d = np.diff(w, axis=0)
    wl = np.sum(np.abs(d), axis=0)                                  # 인접 샘플 차이의 누적: 파형 복잡도
    zc = np.sum(((w[:-1] * w[1:]) < 0) & (np.abs(d) >= th), axis=0)  # 0을 지난 횟수 (작은 흔들림은 무시)
    ssc = np.sum((d[:-1] * d[1:]) < 0, axis=0)                      # 기울기 부호가 바뀐 횟수
    return np.concatenate([rms, mav, wl, zc, ssc])
```

첫 창(1.0~1.2초)의 특징 벡터 (25,). NaN 없음.

	bandpass 1	bandpass 2	bandpass 3	bandpass 4	bandpass 5
RMS	0.0112	0.0071	0.0176	0.0014	0.0053
MAV	0.0085	0.0057	0.0124	0.0011	0.0040
WL	0.6664	0.4799	1.3162	0.0916	0.3099
ZC	3	0	12	0	0
SSC	27	39	43	56	29

ZC 임계값(th)의 효과: th=0이면 잡음의 미세한 부호 변화까지 세서 값이 커진다. 이 데이터는 진폭이 작아(휴식 0.002~0.005, 동작 0.01~0.5) 강의 기본값 th=0.01이 큰 편이며, bandpass 4·5(상완)는 동작 중에도 ZC가 0인 창이 30~50%였다. 이번 주는 기본값을 유지하고, 3주차에서 낮춘 값과 성능을 비교할 예정이다.

```
ZC (5채널)          th=0.01            th=0
  창 1 (동작 시작)   [ 3  0 12  0  0]   [26 28 33 26 23]
  창 10 (동작 중)    [27 27 30  2  0]   [28 28 30 18 26]
  창 19 (동작 끝)    [ 4  4 11  0  0]   [22 29 29 27 23]
```

④ 분할 코드 — 시행 단위로 먼저 나눈 부분

같은 시행에서 나온 19개 창은 서로 절반씩 겹치므로, 창을 만든 뒤 나누면 한 시행의 조각이 학습과 테스트 양쪽에 들어가 정확도가 비현실적으로 높아진다(데이터 누수). 그래서 **시행 번호를 먼저 나누고(아래 (1)), 각 집합 안에서만 창과 특징을 만들**

었다(아래 (2)). 강의 예시는 파일(`files`) 단위로 나누지만, 이 데이터는 파일 하나에 30회 시행이 들어 있어 `trials`의 행 번호(시행)로 나눴다.

```
# (1) 시행 단위로 먼저 나눈다  ← 누수를 막는 핵심
idx_tr, idx_te = train_test_split(trials.index.to_numpy(), test_size=0.2,
                                  stratify=trials["label"], random_state=42)
assert not set(idx_tr) & set(idx_te)   # 겹치는 시행이 없어야 한다

# (2) 각 집합 안에서만 필터 → 자르기 → 윈도우 → 특징
def build(idx: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
    """시행 번호 목록 -> X (시행 수x19, 25), y (창마다의 라벨), g (창이 온 시행 번호)"""
    X, y, g = [], [], []
    for i in idx:
        for wnd in make_windows(cut(preprocess(trial_emg[i]))):
            X.append(td_features(wnd))
            y.append(trials.loc[i, "label"])
            g.append(i)
    return np.stack(X), np.array(y), np.array(g)

Xtr, ytr, gtr = build(idx_tr)
Xte, yte, gte = build(idx_te)
```

```
학습 시행: 1536 / 테스트 시행: 384
Xtr: (29184, 25) / Xte: (7296, 25) (창 수, 25차원)
학습 창 수 / 테스트 창 수 = 4.0 (약 4배면 정상)
```

⑤ 데이터셋 요약

시행 1,920개 중 학습 1,536회(80%) · 테스트 384회(20%). `stratify=label`로 8클래스가 양쪽에 같은 비율로 들어갔고, 시행마다 창 19개가 나오므로 학습 29,184창 · 테스트 7,296창이다.

클래스	학습 (창)	테스트 (창)
30_fist	3,648	912
30_open	3,648	912
30_pronation	3,648	912
30_supination	3,648	912
60_fist	3,648	912
60_open	3,648	912
60_pronation	3,648	912
60_supination	3,648	912
합계	**29,184**	**7,296**

표준화는 `StandardScaler`를 학습셋에만 `fit`하고 테스트셋에는 `transform`만 적용했다.

```
학습셋  평균 (최대 절댓값): 0.0 / 표준편차: 1.0 ~ 1.0
테스트셋 평균 범위: -0.05 ~ 0.08 (정확히 0·1이 아니어도 정상: 학습셋의 통계로 변환했기 때문)
```

저장 파일 `outputs/dataset.npz` (6.0 MB). 3주차 교차검증에서 같은 시행·피험자가 fold 사이로 갈라지지 않도록 창마다의 시행 번호(`gtr/gte`)와 피험자(`subject_tr/te`)를 함께 저장했다.

키	모양	내용
Xtr, Xte	(29184, 25), (7296, 25) float32	표준화된 25차원 특징
ytr, yte	(29184,), (7296,) str	창마다의 클래스 라벨 (예: 30_fist)
gtr, gte	(29184,), (7296,) int	창이 온 시행 번호 (0~1919)
subject_tr, subject_te	(29184,), (7296,) str	창이 온 피험자 (a~h)
feature_names	(25,) str	열 이름 (예: RMS_bandpass 1)

⑥ 설정 근거

윈도우 길이 200 ms

1,000 Hz에서 200샘플이면 RMS·ZC 같은 시간영역 특징이 안정되게 계산되면서도, 실시간 제어에서 허용되는 지연(약 300 ms) 안에 들어온다. 논문과 같은 설정이고, 이 데이터의 동작 구간(약 1.7초)을 여러 창으로 나눠 볼 수 있는 길이다.

오버랩 50%

2초 구간에서 창이 10개가 아니라 19개가 나와 학습 샘플이 거의 두 배가 되고, 100 ms 간격으로 특징이 나와 동작 시작·끝 같은 변화를 놓치지 않는다. 겹친 창은 서로 독립이 아니므로 학습/테스트 분할을 시행 단위로 먼저 해서 누수를 막았다(④).

비고 — 강의 예시와 다르게 한 점

항목	강의 예시	이번 실습
필터	원신호에 적용	데이터가 이미 같은 필터로 처리되어 있음(①). 한 번 더 적용했고 차이는 미미
분석 구간 시작	1.5초 (실습 가정)	1.0초. 1주차 4-2에서 확인한 실제 동작 구간(1.0~2.7초)에 맞춤
분할 단위	파일 = 시행 1개	파일 하나에 30회가 있어 시행(trials 행 번호) 단위로 분할
ZC 임계값	0.01	같은 값 사용. 진폭이 작은 데이터라 상완 채널의 ZC가 0인 창이 많음 → 3주차에서 비교 예정